

MESSAGING AND SCHEDULED JOBS MADE SIMPLE

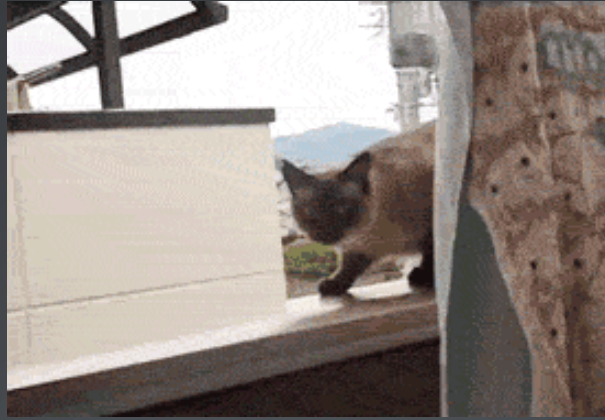
Marek Goldmann



AGENDA

- Introduction to TorqueBox
- Java Message Service
 - Concepts refresh
- TorqueBox Messaging
 - Destination management
 - Sending and receiving messages
 - Message management
- TorqueBox Jobs
 - Scheduled jobs
 - 'At' jobs

A CAT





RUBY APPLICATION SERVER

*“Did he just say **Ruby** at a **Java**
conference?”*

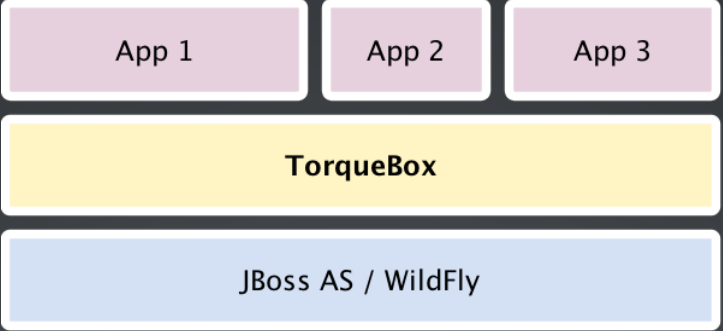
Yep, it's a

RUBY APPLICATION SERVER

based on [JBoss AS](#)







APPLICATION SERVER
IS NOT ONLY
A WEB SERVER

STUFF WE CAN DEPLOY ON TORQUEBOX

- Web applications (both Ruby and Java)
- Services
- Jobs
- Messaging
- A mix of above

```
$ torquebox run
```

JAVA MESSAGE SERVICE

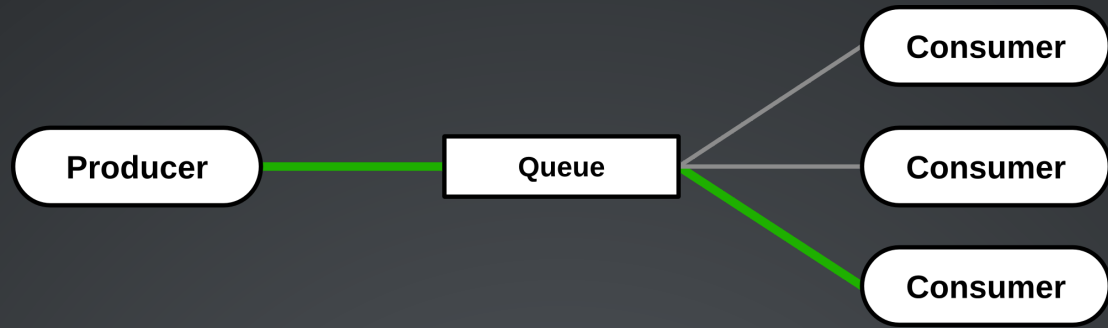
Let's refresh the knowledge!

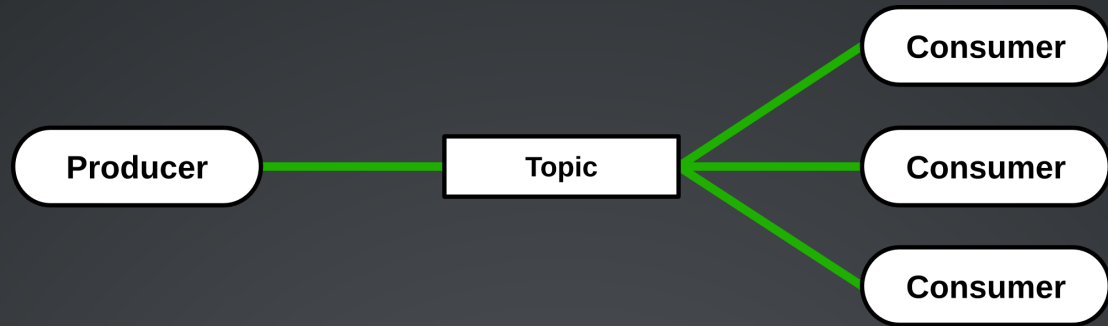
ASYNCHRONOUS!

There are
PRODUCERS
and
CONSUMERS

TWO COMMUNICATION MODELS

1. point-to-point (one to one)
2. publish-subscribe (one to many)





DESTINATIONS

IN THE CODE

```
# Creates and starts a new queue
@queue = TorqueBox::Messaging::Queue.start("/queues/new_queue")

# Creates and starts a new topic
@topic = TorqueBox::Messaging::Topic.start("/topics/new_topic")
```

USING DEPLOYMENT DESCRIPTORS

```
TorqueBox.configure do
  # Creates and starts the queue with the application
  queue '/queues/one'

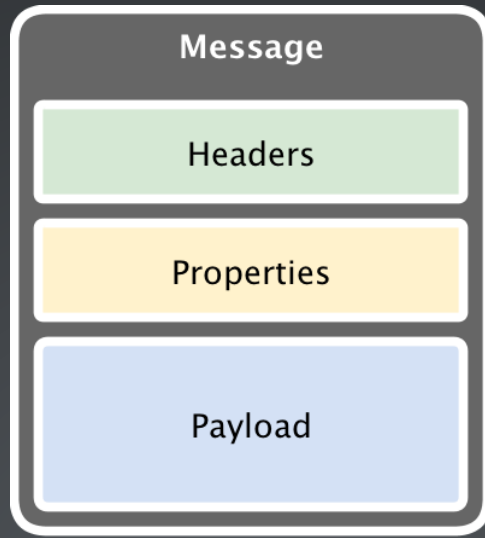
  # Creates and starts the topic with the application
  topic '/topics/one'
end
```

torquebox.rb

DEMO 01

Creating and destroying destinations

MESSAGE



SENDING A SIMPLE MESSAGE

```
# Sending a text message
@queue.publish("Hello!")

# Sending a Ruby Hash
@queue.publish(:type => :car, :price => 20455.73)
```

DEFINING MESSAGE PROPERTIES

```
@queue.publish("Hello!",
:properties => {
  'type' => 'CUCUMBER',    # String
  'count' => 126,          # Integer
  'price' => 4367.23,      # Float
  'valid' => true          # Boolean
}
)
```

RECEIVING A MESSAGE

```
# Watch out – this call can block
# the thread forever if no new messages arrive
message = @queue.receive

# Wait at most 10 seconds for the message
message = @queue.receive(:timeout => 10_000)
```

DEMO 02

Sending and receiving messages

MESSAGE PROCESSOR

```
class SimpleProcessor < TorqueBox::Messaging::MessageProcessor

  def on_message(message)
    puts "Received '#{message}'"
  end

end
```

MESSAGE PROCESSORS USAGE

```
TorqueBox.configure do
  # Define the queue
  queue '/queues/stuff' do
    # Specify the message processor implementation
    processor SimpleProcessor
  end
end
```

torquebox.rb

DEMO 03

Using message processors

MESSAGE FILTERING

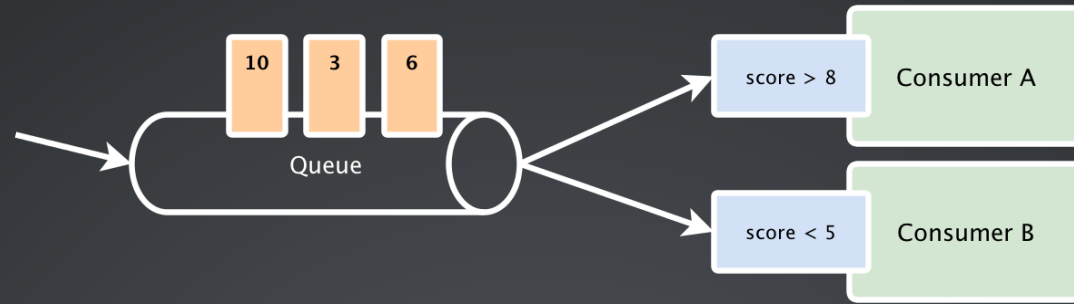
SQL92-like syntax

EXAMPLES

```
type = 'CUCUMBER' AND price < 1.23
```

```
action = 'BUY' AND JMSPriority BETWEEN 5 AND 8
```

```
name LIKE 'MAR%'
```



SYNTAX EXPRESSIONS

- Headers (but not all!)
- Properties
- Operators

RECEIVING A MESSAGE WITH SELECTOR

```
message = @queue.receive(:selector => "type = 'CUCUMBER' AND price < 1.23")
```

MESSAGE SELECTORS IN PROCESSORS

```
TorqueBox.configure do
  # Define the queue
  queue '/queues/selector' do
    # Specify the message processor implementation
    processor SimpleProcessor do
      # Define the filter expression
      selector "score > 5"
    end
  end
end
```

torquebox.rb

DEMO 04

Using message selectors

MESSAGE PRIORITY

By default: 4

Message provider delivers messages with higher priority first

MESSAGE PRIORITY IN TORQUEBOX

- `:low` set to 1
- `:normal` set to 4
- `:high` set to 7
- `:critical` set to 9

EXAMPLES

```
# low priority (1)  
@queue.publish("Is someone there?", :priority => :low)
```

```
# default priority (4)  
@queue.publish("Hello!")
```

```
# critical priority (9)  
@queue.publish("Urgent!", :priority => :critical)
```

DEMO 05

Message priorities

MESSAGE SCHEDULING

By default: publish **NOW!**

EXAMPLES

```
# Message is published immediately (:scheduled set to 0)
@queue.publish("Normal message")

# Message is published 10 seconds after executing the call
@queue.publish("Scheduled message", :scheduled => Time.now + 10)
```

DEMO 06

Message scheduling

SYNCHRONOUS MESSAGING

WAT?

You said that messaging is asynchronous!

EXAMPLE

```
# Publish a message, then wait for reply
answer = @queue.publish_and_receive("Hello")

# Wait for a message and reply
@queue.receive_and_publish do |message|
  message.upcase
end
```

DEMO 07

Synchronous messaging

MESSAGE MANAGEMENT

- Counting messages
- Moving messages to another queue
 - Expiring messages
 - Sending messages to DLQ
- Removing messages

DEMO 08

Message management

QUEUE MANAGEMENT

- Listing destinations
- Pausing / resuming the queue

DEMO 09

Queue management

SCHEDULED JOBS

Things you want to execute sometime in the future

JOB CLASS

```
class SimpleJob
  def run
    # Stuff goes here
  end
end
```

simple_job.rb

SCHEDULING OPTIONS

1. Cron syntax
2. 'At'

Seconds	Minutes	Hours	Day of month	Month	Day of week	Year
*/5	*	*	*	*	?	
0	15	3	1	*	?	

IN THE CODE

```
TorqueBox::ScheduledJob.schedule(  
  "SimpleJob",  
  '*/*5 * * * * ?',  
  :name      => 'simple.job',  
  :description => 'Simple job',  
  :timeout   => '5s',  
  :config    => { 'foo' => 'bar' }  
)
```

DEPLOYMENT DESCRIPTORS

```
TorqueBox.configure do
  job SimpleJob do
    cron '*/* * * * * ?'
  end
end
```

torquebox.rb

DEMO 10

Cron jobs

'AT' JOBS

```
# Run a job every 200 ms for over 5 seconds, from now
TorqueBox::ScheduledJob.at(
  'SimpleJob', :every => 200, :until => Time.now + 5)

# Start in 1 second, then every 200 ms for over 4 seconds
TorqueBox::ScheduledJob.at(
  'SimpleJob', :at => Time.now + 1, :every => 200, :until => Time.now + 5)

# Start in 1 second, then every 200 ms for over 4 seconds
TorqueBox::ScheduledJob.at(
  'SimpleJob', :in => 1_000, :every => 200, :until => Time.now + 5)

# Start in 1 second, then repeat the job 10 times, every 200 ms
TorqueBox::ScheduledJob.at(
  'SimpleJob', :in => 1_000, :repeat => 10, :every => 200)
```

DEMO 11

'At' jobs

DISCLAIMER

All mentioned features require [TorqueBox 3](#)

Or at least latest incrementals

QUESTIONS?

THANKS!

<http://torquebox.org>